View Reviews

Paper ID
717

Paper Title
Efficient Path Query Processing in Relational Database Systems

Track Name
Research Round 2

Reviewer #2 | Regular

Questions

1. Overall Evaluation
Reject

3. Novelty
No novelty

4. Importance: select all that apply:
No impact expected

5. Summary of contribution (in a few sentences)
The paper presents ReCAP, an abstraction to process regular path queries expressed as recursive join queries more efficiently. ReCAP leverages so-called negatively stable constraints to introduce additional filtering conditions that are used to prune infeasible paths early. The experiments demonstrate that the rewritten queries achieve a speed-up in DuckDB.

6. Describe in detail all strong points, labeled S1, S2, S3, etc.
[S1] Regular path queries are important to graph database systems.

[S2] Query optimization is a relevant topic for the SIGMOD audience.

[S3] Code artifacts are provided via GitHub.

7. Describe in detail all opportunities for improvement, labeled O1, O2, O3, etc.
[O1] I found the phasing "implemented in DuckDB" is misleading as I took it to imply that the approach had been implemented within DuckDB's query optimizer. Rather, the demonstration code (`visual_ReCAP`) provides by the authors contains a Python script that transforms regular expressions into NFAs and then injects this information into a generic, parameterized WITH RECURSIVE query. The architecture of the framework needs to be clearly described in the paper as one cannot rely on readers reverse-engineering it

from the source code.

[O2] Optimizing recursive join queries by introducing filters to prune infeasible solutions early is textbook knowledge and therefore not very novel.

[O3] The interesting questions that would need to be answered in order to automate this approach and truly implement it in a database system remain largely unanswered or is left to the user. For example, how would a system identify negatively stable constraints in order to cope with general queries?

[O4] Nevertheless, the results of the limited study that the authors conducted with their prototype clearly indicate optimization potential.

8. Please rank the three most critical strong points and/or opportunities for improvement that led to your overall evaluation rating, e.g., "{S1, O7, O4}"
O1, O2, O3

9. Do you have specific questions to authors to answer in author feedback? Use Q1, Q2, Q3, etc. for questions.
N/A

10. Minor remarks: Use this field to describe minor remarks that are not listed as opportunities for improvement, e.g., to highlight typos, formatting problems, or minor technical issues, labeled M1, M2, M3, etc.
N/A

11. Required changes for revision: List required changes for a revision, if appropriate, by identifying subset of previously specified opportunities for improvement (e.g., O1, O3, O6). Authors can respond to this in their proposed initial revision plan along with the author feedback.
N/A

12. Availability of artifacts: All papers were asked to make their code, data, scripts, and notebooks available anonymously if possible. You can find this information in the paper summary on CMT when clicking on the Paper ID next to the title. The call for papers emphasizes the need for artifacts especially for E&A Papers and DI&DS Papers. Are the artifacts available? Do you have any comments or feedback on the artifacts?
Artifacts are available on GitHub. As I was interested in how this approach works, I had a close look at the Python source code provided by the authors. To my disappointment, the source code clearly shows that the paper talks about a hard-coded prototype that one would use in a preliminary feasibility study to investigate whether there is any potential for research here.

Reviewer #3 | Regular

Questions

1. Overall Evaluation

Minor Revision - An "accept" post revision is expected

3. Novelty

Novel solution

4. Importance: select all that apply:

The paper is likely to influence other research in the community

5. Summary of contribution (in a few sentences)

The authors advance state of the art in processing path queries with complex data constraints by providing an elegant framework for expressing such queries as a combination of WITH RECURSIVE statements in SQL, automata-driven path exploration, and user defined functions which allow early filtering of data constraints along paths. An implementation with a decent experimental evaluation is provided and the work shows marked improvement over similar approaches.

6. Describe in detail all strong points, labeled S1, S2, S3, etc.

S1. The paper studies a problem that is highly relevant in graph databases (detecting paths with complex data conditions), which was mostly studied in a theoretical setting, but no existing solutions tackle the problem in an efficient manner.
S2. The writing is very clear and well presented.
S3. The contribution itself is quite elegant, as it mixes well know ingredients (SQL, automata-driven path exploration and user defined functions) to solve the proposed problem.
S4. The implementation shows marked improvement over existing solutions (in the use case for which it was constructed).

7. Describe in detail all opportunities for improvement, labeled O1, O2, O3, etc.

O1. There is one major point that is not clear from the experiments: how much is the improvement due to the automata-driven exploration and how much is due to early pruning of data constraints? Previous works (in particular [13]) showed that just using the NFA approach to RPQ exploration already provides a non-trivial advantage over existing solutions. Having such an experiment here would allow to confirm this claim. In particular, it would be interesting to see if, when simply the regex part (hopefully on a non-trivial pattern) is explored on paths of increasing length, there is already a significant gain. This should also be leveraged on whether the path is returned or not. Also on how selective the regex itself is.
O2. The experiments are done on graphs of rather modest size. Is there a particular reason for this?

O3. Given that the proposed approach can also return paths, why is this not mentioned. Also, given this, it would be interesting to know what is the memory cost is. Sure, when data constraints filter early the storage overhead would be trivial, but what would happen if many paths need to be stored? This is a more general concern about having only 4 test queries (in particular after reading section 7.4).

8. Please rank the three most critical strong points and/or opportunities for improvement that led to your overall evaluation rating, e.g., "{S1, O7, O4}"
S1, S3, O1

9. Do you have specific questions to authors to answer in author feedback? Use Q1, Q2, Q3, etc. for questions.
N/A

10. Minor remarks: Use this field to describe minor remarks that are not listed as opportunities for improvement, e.g., to highlight typos, formatting problems, or minor technical issues, labeled M1, M2, M3, etc.
Reference usage seems to be slightly off in some places. For instance, in section 5.3 automata models with memory are referenced (probably citations [16] and [36] in the paper), but [21] is used. Likewise, [21] is not mentioned when list post-processing approach is explained initially (Intro), although it makes this argument quite clearly.

11. Required changes for revision: List required changes for a revision, if appropriate, by identifying subset of previously specified opportunities for improvement (e.g., O1, O3, O6). Authors can respond to this in their proposed initial revision plan along with the author feedback.
I would insist on O1 being cleared up. O2 would be ice to explore as well.

Also, the github repo mentions that the implementation does not support all regex, while the paper claims this. This should possibly be cleared up (as I state below, not a major issue).

12. Availability of artifacts: All papers were asked to make their code, data, scripts, and notebooks available anonymously if possible. You can find this information in the paper summary on CMT when clicking on the Paper ID next to the title. The call for papers emphasizes the need for artifacts especially for E&A Papers and DI&DS Papers. Are the artifacts available? Do you have any comments or feedback on the artifacts?
The artifacts are available on an anonymous repository.

One thing of note is that the artifact itself states that only simple regex/NFAs are supported and not all regex. While this is not a major omission, in the paper it is claimed that full support is granted. To be honest, adding this is rather trivial, so I do not understand why it was not done.

Reviewer #4 | Regular

Questions

1. Overall Evaluation

Major Revision - Many revisions required and an "accept" post revision is not certain

3. Novelty

Novel solution

4. Importance: select all that apply:

SIGMOD attendees will learn something interesting from the paper
The paper is likely to influence other research in the community

5. Summary of contribution (in a few sentences)

The authors address a gap in the optimization of regular path queries (RPQs) with constraints on properties over property graphs. That is, SOTA systems for graph databases and queries must effectively rely on "list processing" (e.g., as in Cypher in Neo4j) to collect the relevant properties associated with a path matching the RPQ that can then be checked only after the fact — after the path has been generated — for whether the path's properties satisfy the query's property constraints. That is, their evaluation strategies presently mean finding the paths in the graph matching the regular-path template of the query and checking the property constraints are decoupled. But this means that evaluating RPQs with property constraints can be very expensive: if many paths satisfy the regular-path part of the query but not its property constraints, these paths are found first at great expense, only to be filtered out at the end.
The authors address this gap as follows. First, they define a family of property constraints they call negatively stable. A set of constraints is negatively stable if, for any path that violates the constraints, any extension of the path by appending it logically must also violate the constraints. This covers many property constraints in queries seen in practice, so it is an important class. Such constraints could be evaluated incrementally as paths are grown during the evaluation of the regular path query, eliminating intermediate paths early that fail the property constraints. (Indeed, this is the point of the authors' exercise.) Second, they use recursive SQL as their query language for expressing RPQs with property constraints. This lets users express such queries in a natural and declarative way. The authors build their prototype on DuckDB. Third, They extend the evaluation for such to identify the constraints that are negatively stable to push them into their modified evaluation procedure, which uses them for early pruning.
The paper's contributions are as follows. The authors define the class of negatively-stable property constraints. They define ReCAP as a general framework that can push such constraints into the evaluation for any RDBMS being used for querying property graphs, using recursive SQL to express declarative such queries. They implement ReCAP over DuckDB to demonstrate, and present experiments benchmarking against SOTA graph

database systems, showing extreme speedups for practical, meaningful regular-path queries with property constraints.

6. Describe in detail all strong points, labeled S1, S2, S3, etc.

S1.

The authors define a useful subclass of property constraints that show up commonly in regular-path graph queries that can be used to greatly optimize their evaluation.

S2.

They show in a very general way how to push evaluation of these constraints to be concurrent with path generation to be checked incrementally. They demonstrate the extreme improvements this offers for the evaluation of such queries over the SOTA.

While I find the approach to be quite obvious, this is a gap that has been clearly missing in the SOTA for graph query processing. And the authors define it, put the pieces together, and make it work.

7. Describe in detail all opportunities for improvement, labeled O1, O2, O3, etc.

O1. MISSING A MORE STRAIGHTFORWARD APPROACH?

The motivating example in Listing 1 expresses the property constraint that the times be monotonically non-decreasing in the WHERE clause. But that would be very straightforward to express in the WHERE of recursive construction, where it would be defined logically as an incremental check on each recursive step expansion, so actually evaluated incrementally, ReCAP or not. And this is also the case for the min-max check. This too can be written as a constraint in the RECURSIVE part, despite the authors' claim, "For the max-min difference on the transaction amounts, the situation is even more challenging, because computing a path aggregate like the maximum across all edges indeed requires knowing all edges." I mean, the path is being literally accumulated, so min and max of the pre-path can be incrementally accumulated and checked. And so the distinctness check for edges on the path can also be expressed in the RECURSIVE part as an "incremental" check. Note that this can be always done for ANY negatively stable property constraint.

The authors argue in the paper that, "The reason for the poor scalability does not lie in the ordering of the clauses: SQL and Cypher are declarative languages, and the order of clauses does not imply evaluation order." In principle, this is true. But in practice, it is not true in RDBMSs' evaluation of recursive SQL queries. The table is grown BFS as a bottom-up fixpoint computation. So the checks for negatively-stable property constraints can just be piggy-backed onto this with no extra machinery as in ReCAP for the evaluation needed.

The authors might argue in rebuttal that, well, forcing the user to meticulously encode the constraints in the RECURSIVE part instead of however they might want to write them naturally violates the declarative ideal. I agree. And this could perhaps be addressed by a rich set of rewrite transformations that could "push" the constraints into the RECURSIVE statement. But if all the machinery of ReCAP is really providing it to protect a weak veneer

of declarativeness, its contribution is greatly diminished.
I am not sure how this could be properly addressed in revision of the paper. Perhaps the machinery of ReCAP provides better pruning in some cases than pushing the constraints into the RECURSIVE part and having them evaluated as part of the recursive expansion? I do not believe this to be the case, but if that is shown, it changes the game.
The authors could benchmark the queries rewritten with the constraints embedded in the RECURSIVE part then run on DuckDB to compare against ReCAP. If ReCAP shows better performance, that could justify the approach.
Assuming this can be addressed, I will proceed to my other recommendations for improvement.
O2. DISCUSS LIMITATIONS AND POSSIBLE EXTENSIONS
The paper does not discuss limitations of their approach. This would be useful for researchers in the area. A main limitation is that incremental checks of the intermediate paths as defined only works if the paths are being grown incrementally from start of the expression to accepting state. And yes, that is how recursive SQL evaluation is presently done (and as it is awkwardly defined in the SQL standards). So, for any evaluation that is limited to this approach, which most presently are, the authors' optimization is quite effective. But many richer evaluation strategies are being explored by researchers. Path exploration for RPQs can grow intermediate paths in segments, taking advantage of data statistics for cost-based optimization where best to start such segments. How could the authors' approach be integrated with such other optimization strategies?
O3. PRESENTATION AND RELATED WORK
Listing 1 and 2 are wrong? The SQL standards for recursion are specified by "base case" UNION ALL / DISTINCT "inductive case". The base-case queries do not have a FROM and so the column "s", for example, in line 149 is undefined. In Listing 1, this should be FROM Accounts and "s" should be ID, no? Perhaps this was done for brevity, but I think this will throw readers.
There is deep related work in this area. The paper's Section 8 could do a better job relating it to their work. I happened to recently read papers [59, 60], and how they are described here is simply wrong. This worries me for how other related work may be mischaracterized? This needs a thorough review.

8. Please rank the three most critical strong points and/or opportunities for improvement that led to your overall evaluation rating, e.g., "{S1, O7, O4}"
O1

9. Do you have specific questions to authors to answer in author feedback? Use Q1, Q2, Q3, etc. for questions.
How would you address O1 in revision?

10. Minor remarks: Use this field to describe minor remarks that are not listed as opportunities for improvement, e.g., to highlight typos, formatting problems, or minor technical issues, labeled M1, M2, M3, etc.

For whatever reason, "SOTA" has become the standard abbreviation for "State Of The Art". Thus, "SOA" is jarring and will not be readily understood by the reader.

11. Required changes for revision: List required changes for a revision, if appropriate, by identifying subset of previously specified opportunities for improvement (e.g., O1, O3, O6). Authors can respond to this in their proposed initial revision plan along with the author feedback.
O1, O2, O3

12. Availability of artifacts: All papers were asked to make their code, data, scripts, and notebooks available anonymously if possible. You can find this information in the paper summary on CMT when clicking on the Paper ID next to the title. The call for papers emphasizes the need for artifacts especially for E&A Papers and DI&DS Papers. Are the artifacts available? Do you have any comments or feedback on the artifacts?
Yes, made well available.

Reviewer #5 | Regular

Questions

1. Overall Evaluation
Major Revision - Many revisions required and an "accept" post revision is not certain

3. Novelty
Novel solution

4. Importance: select all that apply:
The paper clearly advances the state of the art in its field

5. Summary of contribution (in a few sentences)
This paper proposes an efficient approach to solve a class of path queries containing a regular expression and a Boolean predicate on all paths of a property graph G in relational database systems. The key observation is
that early filtering can be done when a partial result path makes the Boolean predicate false. The author calls such a Boolean predicate a negatively stable constraint: whenever a predicate makes a path false it will makes all extensions of the path false. The paper then implements a Non-Deterministic Automaton (NFA) as a relational table, and defines the RECAP for a path query as a pair (N,A) where N is the NFA and A a selective aggregate (a set of user-defined functions) for the Boolean predicate on the NFA. This way, the RECAP can be executed in a relational database with early filtering of Boolean expression as user-defined functions on the partial result paths.
The author designed four example queries and experimented with leading graph as well

as relational databases. The performance results are impressive.

6. Describe in detail all strong points, labeled S1, S2, S3, etc.

S1. The paper identifies an interesting class of path queries, which contain complex Boolean predicates such as temporal constraints and aggregates.

S2. The proposed RECAP approach is effective in filtering out partial result paths early by applying a subset of Boolean predicates.

S3. The proposed approach is implemented in a relational database using a relational table to represent an NFA and user-defined functions for Boolean predicates.

S4. The authors compared RECAP with leading graph and relational databases.

S5. The paper is well-written with representative example queries.

7. Describe in detail all opportunities for improvement, labeled O1, O2, O3, etc.

O1. While the problem of complex path queries is interesting, the proposed approach and the experimental evaluation are based on a few representative example queries. I wonder if any existing workloads contain the target query class, and how much the evaluation of these complex path queries affects the overall performance of graph database queries such as those in the LDBC benchmark. It would strengthen the paper greatly if you can revise an LDBC query to include an example path query and measure the overall performance. The point is to show the proposed techniques work for general queries, not just a few examples, and that the techniques improve the overall query performance.

O2. Some definitions may seem inconsistent with each other in terminologies. For example, Definition 4 contains a Boolean function, Definition 7 defines the Boolean function in Definition 4 as a negatively stable constraint, and then Lemma 1 states the Boolean function as a conjunction of a number of Boolean functions. So we have functions, constraints, and conjunctive predicates. Does it mean the Boolean function in Definition 4 a conjunctive Boolean predicate? Do you handle disjunctions? Please clean up the formalization and make it rigorous and consistent.

O3. RECAP requires the user to write the regular expression and the selective aggregate, which can be complex as the selective aggregate is based on the NFA and contains several functions. Figure 6 shows that the NFA is specified by the user, and converted into relational tables by the system, even though the caption says "the system generates the NFA". Section 5.2 says the user specifies the regular expression and writes Python code for the user-defined functions, and that the system generates the NFA. Please clarify the requirements for the user to make use of the proposed techniques and discuss issues

and potential alternatives for reducing the user's workload and making the system more automatic.

8. Please rank the three most critical strong points and/or opportunities for improvement that led to your overall evaluation rating, e.g., "{S1, O7, O4}"
{S2,S3, O1}

9. Do you have specific questions to authors to answer in author feedback? Use Q1, Q2, Q3, etc. for questions.
Please see questions in O1, O2, O3 and M1-M9.

10. Minor remarks: Use this field to describe minor remarks that are not listed as opportunities for improvement, e.g., to highlight typos, formatting problems, or minor technical issues, labeled M1, M2, M3, etc.
M1. The abstract states that the paper proposed a notion of negatively stable constraints and RECAP, an "abstraction that leverages it for holistically specifying early filtering opportunities in path queries". The "it" in this sentence refers to negatively stable constraints?

M2. Also in the abstract, it states that a RECAP is created by (1) specifying the regular expression for the label constraint (as in
the MATCH clause of Cypher or GQL), (2) defining functions to manage
state (like for a user-defined aggregate), and (3) defining a Boolean
function to identify early filtering opportunities on the state. Is the "state" here means the NFA state? What kind of Boolean functions does RECAP define to identify early filtering opportunities on the state? At this point, the reader does not know about the NFA or the selective aggregate yet.

M3. Section 2.
The first paragraph says the paper ignores vertex labels and properties and that extensions to vertices would be straightforward. Any extensions have been made?

M4. Definition 4. A path query is defined to contain a regular expression and a Boolean function. Theoretically, you can write a regular expression in a Boolean function. Maybe you want to specify that the Boolean function is a conjunctive predicate on labels and properties?

M5. Definition 6. The term "Doomed" doesn't sound scientific.

M6. Lemma 1. should "if any" be "if some"?

M7. Definition 8 defines a selective aggregate based on an NFA and an edge type E to be

a tuple consisting of six elements (one dictionary and five functions), which looks complicated. Also, these elements do not correspond to the name "selective aggregate". Maybe give another name that reflects the contents - user defined functions over the NFA and the edge label?

M8. Definition 9 further defines the RECAP for a path query Q on a graph to be the NFA and the selective aggregate. It seems like a physical query plan, not really an "abstraction", as it needs to construct the NFA and then the selective aggregate functions based on the NFA states.

M9. Figure 8 compares RECAP-standard with RECAP-optimized, and Figure 9 compares RECAP with other systems. So the RECAP in Figure 9 is the RECAP-optimized? If so, on Q2, RECAP-optimized is 1-2 orders of magnitude faster
than RECAP-standard (Figure 8), and RECAP is 6-12 times faster than the other systems in (Figure 9). The paper says Q2 does not have early filtering, then the differences between RECAP-standard and RECAP-optimized are the piggybacking color checks? How is RECAP-standard compared with the other systems in performance?

11. Required changes for revision: List required changes for a revision, if appropriate, by identifying subset of previously specified opportunities for improvement (e.g., O1, O3, O6). Authors can respond to this in their proposed initial revision plan along with the author feedback.
O1-O3

12. Availability of artifacts: All papers were asked to make their code, data, scripts, and notebooks available anonymously if possible. You can find this information in the paper summary on CMT when clicking on the Paper ID next to the title. The call for papers emphasizes the need for artifacts especially for E&A Papers and DI&DS Papers. Are the artifacts available? Do you have any comments or feedback on the artifacts?
The paper summary in CMT contains a URL to the artifcats. The Github seems good. There is no explict RECAP-standard versus RECAP-optimized differentiation in the directory though.